Shamir Secret Sharing

Distributed System Implementation

Team-10 members:

Karim Khabibrakhmanov : k.khabibrakhmanov@innopolis.university
Bogdan Zinchenko : b.zinchenko@innopolis.university
Artur Aipov : a.aipov@innopolis.university
Nikolay Taran : n.taran@innopolis.university
Ruslan Ustinov : r.ustinov@innopolis.university

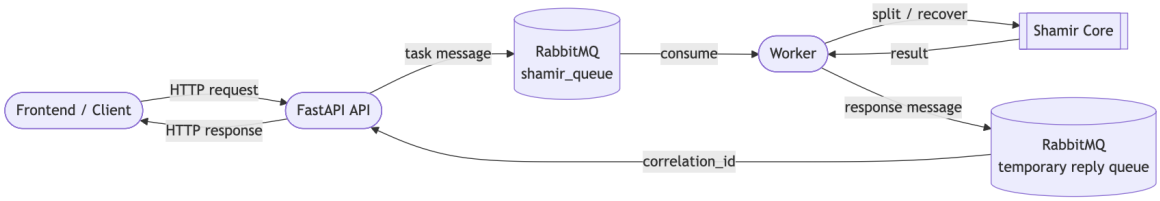
Introduction

We implemented Shamir Secret Sharing for splitting a secret into multiple parts and reconstructing it from a threshold number of shares.

The goal was not only to implement the algorithm, but also to build a small distributed system using concepts from DNP.

Methods

Architecture



The API does not perform computations directly. Instead, it sends tasks to RabbitMQ, where a worker processes them. This simulates real distributed systems with separation of concerns.

Why Distributed

We intentionally avoided a simple synchronous design. Instead, we used RabbitMQ to practice:

- async processing

- queue-based communication
- service separation

How Shamir Works

Idea

We represent the secret as a value s and embed it into a polynomial:

$$f(x) = s + a_1 x + a_2 x^2 + \ldots + a_{k-1} x^{k-1}$$

$$share_i = (x_i,\ f(x_i) \text{ mod } p)$$

$$s = f(0)$$

The secret is never directly shared. Each share is just a point on the same polynomial.

Encoding (Splitting Secret)

We generate a random polynomial where:

- constant term = secret
- other coefficients = random
- all operations are done in finite field Z_p

```
prime, coefficients = generate(secret_int, threshold)

for x in range(1, total_shares + 1):
    y = calculate_polynomial(coefficients, x, prime)
```

Each computed point (x, y) becomes a share.

Decoding (Recovering Secret)

To recover the secret, we reconstruct the polynomial at $x = 0$ using Lagrange interpolation.

$$f(0) = \sum_{i=1}^{k} y_i \cdot \prod_{j \neq i} \frac{-x_j}{x_i - x_j} \text{ mod } p$$

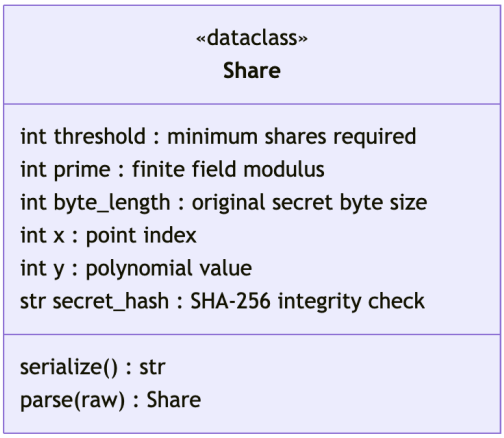
```python
secret = 0
for i, (x_i, y_i) in enumerate(points):
    numerator = 1
    denominator = 1

    for j, (x_j, _) in enumerate(points):
        if i != j:
            numerator = (numerator * (-x_j)) % prime
            denominator = (denominator * (x_i - x_j)) % prime

    basis = numerator * pow(denominator, -1, prime)
    secret = (secret + y_i * basis) % prime
```

We compute the value of the polynomial at zero without ever reconstructing the full polynomial explicitly.

Why It Works

- Any k points uniquely define the polynomial
- Fewer than k points give no information about s
- All computations are performed modulo a large prime p

Share Structure

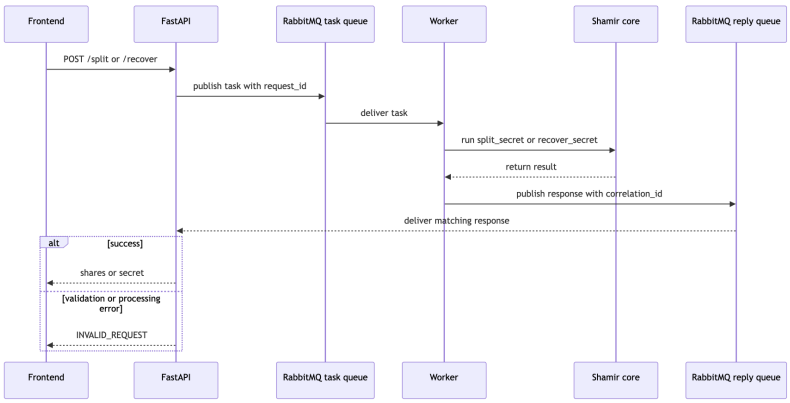


Each share contains all required metadata to reconstruct the secret:

- threshold
- prime (finite field)
- byte length
- x, y (point on polynomial)
- hash (SHA-256 for integrity)

The hash ensures that reconstructed data is valid and not corrupted.

Processing Flow



- Client sends request to API
- API publishes task to queue
- Worker consumes and processes it
- Result is sent back via reply queue
- API returns response to client

Logging (Observability)

The system uses structured logging to trace each request across components. Every log entry includes:

- request_id — unique request identifier
- operation — split / recover
- event — current stage

Successful request flow

```
INFO api.router request_id=abc123 operation=split event=received
   threshold=3 total_shares=5 secret_length=5
INFO broker.rabbit request_id=abc123 operation=split event=
   published_to_queue threshold=3 total_shares=5
INFO broker.worker request_id=abc123 operation=split event=
   worker_started threshold=3 total_shares=5
INFO shamir.encoder operation=split event=core_generated threshold
   =3 total_shares=5 shares_count=5
INFO broker.worker request_id=abc123 operation=split event=
   worker_finished success=true shares_count=5
INFO api.router request_id=abc123 operation=split event=
   response_sent status=200 shares_count=5
```

Error scenario (recover)

```
WARNING broker.worker request_id=xyz789 operation=recover event=
   worker_finished success=false reason=insufficient_shares
WARNING api.router request_id=xyz789 operation=recover event=
   failed reason=insufficient_shares
```

Logs reflect the distributed nature of the system:

- API logs request lifecycle
- broker logs message routing
- worker logs computation results

This makes debugging and failure analysis straightforward.

Validation & Error Handling

All user-facing errors are returned in a unified format.

```
{
  "code": "INVALID_REQUEST",
  "message": "error text",
  "request_id": "..."
}
```

Errors are grouped into three categories.

Input
- secret empty
- threshold < 2
- total_shares < 2
- threshold > total_shares
- invalid body

Reconstruction
- insufficient shares
- malformed share
- duplicate x
- inconsistent shares
- invalid shares

Infrastructure
- worker timeout

Hash validation ensures that even mathematically valid but corrupted results are rejected.

Validation Checklist

- SHA-256 hash is used to verify integrity of reconstructed secrets
- Reconstruction with insufficient shares fails
- All operations are logged for auditability

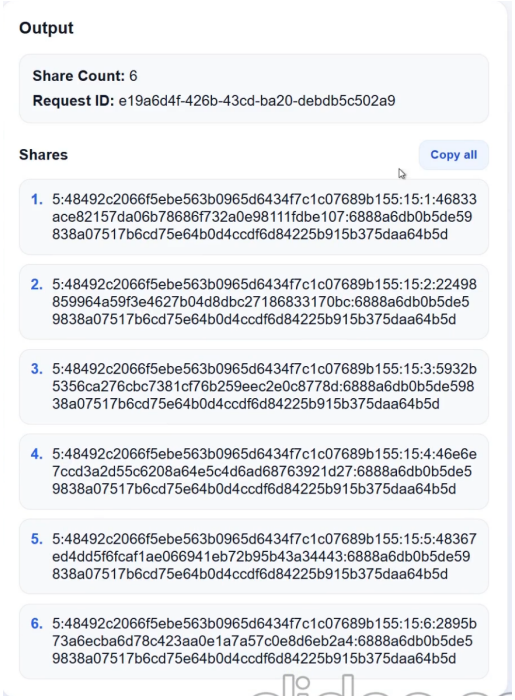
Results

Success

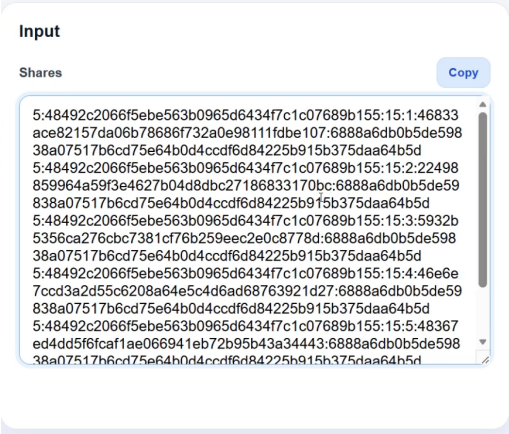
This section demonstrates the system in action for successful split and recovery scenarios. The screenshots below show the user input, the generated shares, and the correct reconstruction flow.

Initial split request

Secret is split into valid shares

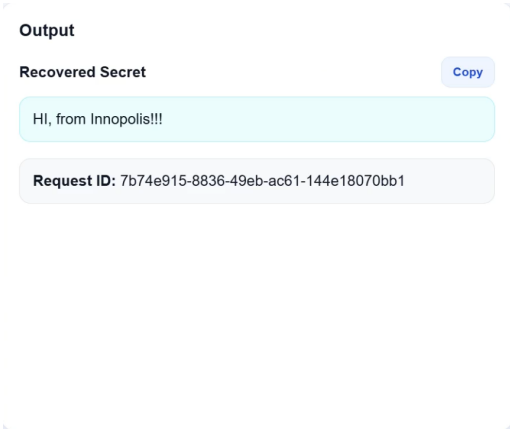


Successful split result



Valid shares entered for recovery

Shares are processed by the backend

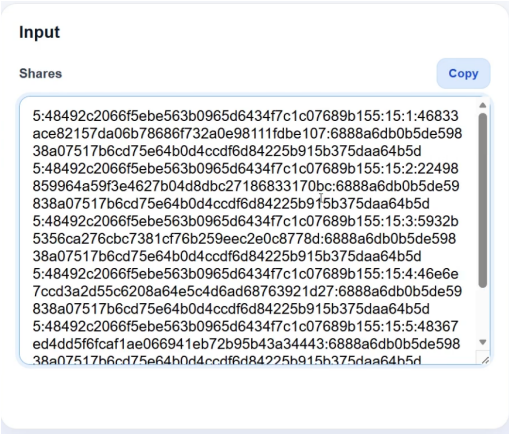


Secret successfully reconstructed

The successful scenarios confirm that the frontend, API, RabbitMQ broker, worker, and Shamir core interact correctly. The split flow returns a set of serialized shares, while the recover flow reconstructs the original secret when enough valid shares are provided.

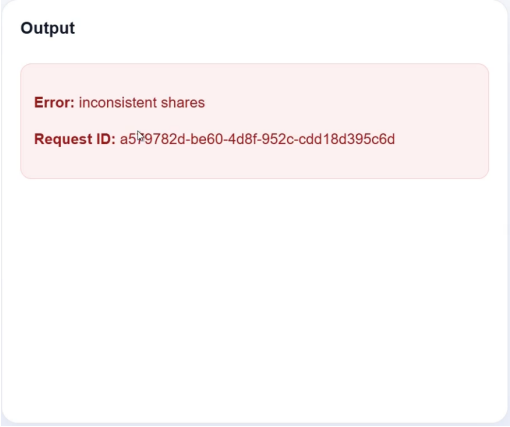
Error

This section illustrates failure scenarios and demonstrates that the system correctly detects corrupted or invalid input. In the example below, one symbol is removed from a share before reconstruction, which causes the backend to reject the request.



Original valid shares

Delete one symbol from a share



Recovery fails with validation error

These error scenarios show that the system not only handles successful requests, but also validates malformed or corrupted input. This is important for reliability, because mathematically inconsistent or tampered shares must not reconstruct a secret silently.

Discussion

At first, we focused mainly on making the project correct from the DNP perspective. That is why we chose a distributed design with API, RabbitMQ, and worker separation instead of a simple synchronous solution.

At the same time, we realized that the mathematical correctness of the Shamir algorithm was just as important. A distributed architecture is not useful if the reconstruction logic, validation rules, or share format are incorrect.

It was also important that all parts of the system were tightly coordinated. The frontend, API, broker, worker, and core had to exchange clear and structured requests and responses so that the system worked smoothly and predictably.

While analyzing the visualization resource in Reference [2], we noticed a weakness: after deleting several symbols from shares, the original plaintext could still often be partially guessed. In our project, this issue was eliminated. Corrupted or inconsistent shares do not reveal the secret and instead produce a controlled error such as `Inconsistent shares`.

Overall, the project showed that a good DNP solution requires both a well-coordinated distributed architecture and a mathematically correct cryptographic core.

Conclusion

Team Work

- **Karim Khabibrakhmanov** — architecture, RabbitMQ, report
  Designed system architecture, implemented message broker flow and overall system design.
- **Bogdan Zinchenko** — core logic
  Implemented polynomial generation and Shamir core logic.
- **Artur Aipov** — API
  Developed FastAPI backend and request handling.
- **Nikolay Taran** — core logic
  Worked on encoding/decoding and validation logic.
- **Ruslan Ustinov** — UI
  Implemented frontend and integration with backend.

We combined cryptography with distributed systems.

The project helped us understand how real systems separate computation and communication.

References

The following resources were used to understand the theoretical background of Shamir Secret Sharing, cross-check the algorithmic intuition, and support implementation decisions.

[1] Adi Shamir. How to Share a Secret. Programming Techniques, edited by R. Rivest. Massachusetts Institute of Technology. https://web.mit.edu/6.857/OldStuff/Fall03/ref/Shamir-HowToShareASecret.pdf

[2] Ian Coleman. Shamir Secret Sharing Visualization. Interactive online visualization of the algorithm. https://iancoleman.io/shamir/

[3] YouTube. Protecting Your Secrets With Polynomials — Shamir's Secret Sharing. Video explanation of the idea behind secret sharing with polynomials. https://youtu.be/$_EHMd8gpApo?si = D8hQsjuNVlk6klnh$